

ABSTRACT

ABSTRACT

Recommender systems play a crucial role in modern digital platforms by predicting user preferences and providing personalized suggestions. The project titled “Collaborative Filtering-Based Book Recommender System Using Machine Learning” aims to address the challenge of information overload in the domain of books, where users often find it difficult to identify relevant content from a vast collection.

The proposed system utilizes collaborative filtering techniques, which operate on the principle that users with similar interests in the past are likely to share similar preferences in the future. By analyzing user ratings and interactions, the system identifies similarities between books using cosine similarity and generates accurate recommendations. This approach does not require detailed knowledge of book content, making it efficient and scalable for large datasets.

To enhance recommendation quality, the system considers both explicit data (such as user ratings and preferences) and implicit data (such as user interactions and browsing behavior). Additionally, the system integrates Natural Language Processing (NLP) techniques to generate concise summaries of books, helping users quickly understand the content before making decisions.

Despite challenges such as cold start problems, sparsity, and scalability, the system is designed to deliver efficient and meaningful recommendations. Implemented as a Flask-based web application, it provides a user-friendly interface for registration, login, and personalized recommendation features.

Overall, the system improves user experience by reducing manual effort, increasing recommendation accuracy, and enabling efficient discovery of books, demonstrating the effective application of machine learning in real-world recommendation systems.

INTRODUCTION

1. INTRODUCTION

1.1 INTRODUCTION

A recommender system, also known as a recommendation system, is a subclass of information filtering systems that aims to predict the preference or rating a user would give to an item. These systems have become an essential part of modern digital platforms and are widely used in commercial applications to provide personalized suggestions. Recommender systems are commonly seen in services such as Netflix, YouTube, and Spotify for content recommendations, Amazon for product suggestions, and social media platforms like Facebook and Twitter for personalized feeds. They help users discover relevant items efficiently from large collections of data.

With the rapid growth of digital information, especially in the domain of books, users are exposed to a vast number of options across different genres, authors, and languages. This abundance of information often leads to difficulty in selecting relevant content, a problem commonly referred to as information overload. Traditional systems rely on general popularity or manual search, which does not consider individual user preferences, resulting in poor user experience and reduced engagement.

To overcome these limitations, recommender systems use various techniques to analyze user behavior and generate personalized recommendations. One of the most widely used approaches is collaborative filtering. Collaborative filtering is based on the assumption that users who have shown similar interests in the past are likely to have similar preferences in the future. This approach generates recommendations by identifying users or items with similar rating patterns and forming a neighborhood of similarity.

Collaborative filtering techniques can be broadly classified into user-based and item-based approaches. User-based collaborative filtering identifies users with similar preferences and recommends items liked by those users. In contrast, item-based collaborative filtering focuses on finding similarities between items and recommending items that are similar to those previously liked by the user. In this project, item-based collaborative filtering is primarily used due to its better performance and scalability when handling large datasets.

One of the major advantages of collaborative filtering is that it does not require detailed knowledge about the content of the items. Instead, it relies solely on user interaction data such as ratings and behavior. This makes it highly effective for recommending complex items such

as books, movies, and music. Various algorithms are used to measure similarity in recommender systems, including cosine similarity, Pearson correlation, and k-nearest neighbors (KNN). In this project, cosine similarity is used to compute the similarity between books and generate accurate recommendations.

Recommender systems collect data in two main ways: explicit and implicit data collection. Explicit data includes direct user inputs such as ratings, reviews, and preferences. Implicit data includes user behavior such as browsing history, clicks, purchase records, and interaction time. Both types of data help in building user profiles and improving recommendation accuracy.

Despite their advantages, collaborative filtering approaches face several challenges. The cold start problem occurs when there is insufficient data for new users or items, making it difficult to generate recommendations. Scalability becomes an issue when dealing with large datasets containing millions of users and items, requiring significant computational resources. Data sparsity is another challenge, where only a small portion of items are rated, leading to incomplete information.

The project titled “Collaborative Filtering-Based Book Recommender System Using Machine Learning” aims to develop an efficient and intelligent system to provide personalized book recommendations. The system analyzes user preferences and ratings to identify similarities between books using machine learning techniques. By applying cosine similarity, it generates relevant recommendations tailored to individual users.

In addition to recommendation functionality, the system incorporates Natural Language Processing (NLP) techniques to generate summaries of books. This feature allows users to quickly understand the content of a book before making a decision, thereby improving usability and user satisfaction.

The main objective of this project is to enhance user experience by providing accurate and personalized recommendations, reducing the effort required to search for books, and enabling efficient discovery of relevant content. The system demonstrates the practical application of machine learning techniques in solving real-world problems and highlights the importance of recommender systems in modern digital environments.

LITERATURE SURVEY

2. LITERATURE SURVEY

2.1 LITERATURE REVIEW

Recommender systems have been widely studied in the field of machine learning and data mining due to their importance in providing personalized suggestions. Over the years, several researchers have proposed different techniques to improve the accuracy, scalability, and efficiency of recommendation systems, particularly in collaborative filtering.

One of the earliest and most influential works in this domain is by G. Linden, B. Smith, and J. York (2003), who introduced the concept of item-to-item collaborative filtering in Amazon's recommendation system. Their approach focused on analyzing user purchase behavior to recommend items based on similarities between products. This method significantly improved scalability and became a foundation for many modern recommender systems [1].

J. L. Herlocker, J. A. Konstan, and J. Riedl (2004) contributed to the evaluation of collaborative filtering systems by proposing various techniques to measure recommendation accuracy. Their work emphasized the importance of user similarity measures and introduced metrics such as precision, recall, and mean absolute error for evaluating system performance [2].

Y. Koren, R. Bell, and C. Volinsky (2009) proposed matrix factorization techniques for recommender systems, which improved both accuracy and scalability. Their approach decomposed large user-item matrices into smaller latent feature representations, allowing the system to capture hidden patterns in user preferences. This method was widely adopted in large-scale applications such as the Netflix Prize competition [3].

X. Su and T. M. Khoshgoftaar (2009) provided a comprehensive survey of collaborative filtering techniques, including user-based and item-based methods. Their study highlighted the strengths and weaknesses of each approach and discussed challenges such as data sparsity, cold start problems, and scalability issues [4].

J. Bobadilla, F. Ortega, A. Hernando, and A. Gutiérrez (2013) presented an extensive overview of recommender systems and proposed improvements in collaborative filtering techniques. Their work focused on enhancing recommendation quality by incorporating hybrid methods that combine multiple approaches [5].

Despite significant advancements, existing systems still face limitations. Many traditional recommendation systems rely heavily on user ratings and fail to perform effectively when data is sparse or unavailable. Moreover, most systems lack features such as automatic content summarization, which can help users quickly understand the recommended items.

The proposed system, “Collaborative Filtering-Based Book Recommender System Using Machine Learning,” builds upon these existing research works by combining item-based collaborative filtering with cosine similarity to generate accurate recommendations. Additionally, it integrates NLP-based summary generation to enhance user experience. By addressing some of the limitations of existing systems, the proposed model aims to provide a more efficient and user-friendly recommendation platform.

SYSTEM STUDY

3. SYSTEM STUDY

3.1 FEASIBILITY STUDY

The feasibility of the project is analyzed in this phase and a general plan is prepared for the development of the system along with basic cost estimation. During system analysis, the feasibility study of the proposed system is carried out to ensure that the system is practical and does not create unnecessary burden in terms of cost, resources, or usability. For feasibility analysis, it is essential to understand the major requirements of the system and evaluate whether they can be fulfilled efficiently.

The proposed system titled “Collaborative Filtering-Based Book Recommender System Using Machine Learning” is evaluated based on three key considerations:

- ECONOMICAL FEASIBILITY ○
- TECHNICAL FEASIBILITY ○
- SOCIAL FEASIBILITY

3.2 ECONOMICAL FEASIBILITY

This study is carried out to evaluate the economic impact of the system. The total cost involved in the development and implementation of the system must be justified and kept within budget limits. The proposed system is highly economical as it uses open-source technologies such as Python, Flask, and machine learning libraries like Pandas, NumPy, and Scikit-learn.

Since these tools are freely available, there is no requirement for expensive software licenses. The system can be developed using a standard computer without the need for high-end infrastructure. Only minimal costs may be involved in system setup and maintenance. Therefore, the overall expenditure is low, making the system economically feasible.

3.3 TECHNICAL FEASIBILITY

This study is carried out to check whether the technical requirements of the system can be met with the available resources. The proposed system is developed using widely used and wellsupported technologies such as Python for programming and Flask for web application development. Machine learning techniques such as collaborative filtering and cosine similarity are implemented using standard libraries.

The system does not require high computational power or advanced hardware. It can be executed efficiently on basic systems with minimal configuration. The technologies used are

easy to understand and implement, and sufficient documentation and community support are available. Hence, the system does not impose heavy demands on technical resources and is technically feasible.

3.4 SOCIAL FEASIBILITY

Social feasibility is carried out to determine the level of acceptance of the system among users. The system is designed with a simple and user-friendly interface, allowing users to easily register, log in, and receive book recommendations based on their preferences.

The system reduces the effort required for searching books and provides personalized suggestions along with summaries, making it highly beneficial for users. Proper guidance and minimal training are sufficient for users to interact with the system effectively. The users are expected to accept the system as it improves convenience and enhances their overall experience.

Thus, the proposed system is socially feasible as it meets user expectations and encourages user interaction.

SYSTEM SPECIFICATIONS

4. SYSTEM SPECIFICATIONS

4.1 SYSTEM REQUIREMENTS

4.1.1 HARDWARE SYSTEM CONFIGURATIONS:

- **System** : Intel Core i3/i5 Processor or above
- **Processor** : Minimum 2.0 GHz
- **RAM** : 4 GB (Minimum)
- **Hard Disk** : 20 GB Free Space
- **Keyboard** : Standard Windows Keyboard
- **Mouse** : Two or Three Button Mouse
- **Monitor** : SVGA / LCD Monitor

4.1.2 SOFTWARE REQUIREMENTS:

- **Operating System** : Windows 10 / Windows 11
- **Programming Language** : Python
- **Frontend** : HTML, CSS, JavaScript
- **Backend** : Flask (Python Web Framework)
- **Database** : SQLite
- **IDE** : Visual Studio Code / PyCharm
- **Libraries Used** :
 - Pandas (Data Processing)
 - NumPy (Numerical Computation)
 - Scikit-learn (Machine Learning)
 - Pickle (Model Storage)
- **Browser** : Google Chrome / Microsoft Edge

SYSTEM ANALYSIS

5. SYSTEM ANALYSIS

5.1 EXISTING SYSTEM

In the current digital environment, a large number of online platforms provide access to books across various genres, authors, and languages. However, most existing systems rely on traditional recommendation approaches such as popularity-based filtering, keyword-based search, and manual browsing. These methods are limited in their ability to provide personalized recommendations to users.

In popularity-based recommendation systems, books are suggested based on overall ratings, number of purchases, or trending status. While this approach may highlight popular books, it does not consider individual user preferences, interests, or reading history. As a result, users often receive the same recommendations regardless of their unique tastes.

Another commonly used approach in existing systems is keyword-based searching. In this method, users are required to manually enter keywords, author names, or categories to find books. Although this provides some level of control, it is time-consuming and depends heavily on the user's ability to specify accurate search terms. Moreover, it does not guarantee that the results will match the user's preferences.

Some systems also use basic filtering techniques, but they often lack intelligence and adaptability. These systems do not learn from user behavior over time and fail to update recommendations dynamically. As a result, they are unable to provide accurate and meaningful suggestions.

Furthermore, many traditional systems do not incorporate machine learning techniques, which limits their ability to analyze large datasets and identify complex patterns in user behavior. They also lack features such as Natural Language Processing (NLP), which can help users understand the content of books through summaries or descriptions.

Another major limitation of existing systems is the absence of personalization. Since these systems do not consider user-specific data, they fail to provide customized recommendations. This leads to reduced user engagement and satisfaction.

Additionally, users are required to spend a significant amount of time browsing through multiple options to find suitable books. This not only reduces efficiency but also creates frustration, especially when dealing with large datasets.

5.1.1 DISADVANTAGES OF EXISTING SYSTEM

- Lack of personalization in recommendations
- Low accuracy due to generic suggestions
- Time-consuming manual search process
- Inability to adapt based on user behavior
- No integration of NLP for content understanding
- Limited scalability for large datasets
- Poor user experience and reduced engagement

5.2 PROPOSED SYSTEM

To overcome the limitations of existing systems, the proposed system titled “Collaborative Filtering-Based Book Recommender System Using Machine Learning” is designed to provide personalized, efficient, and intelligent book recommendations.

The proposed system uses collaborative filtering techniques, which are based on the principle that users with similar interests in the past are likely to have similar preferences in the future. Instead of relying on general popularity, the system analyzes user ratings and interactions to identify relationships between users and books.

In this system, item-based collaborative filtering is used to determine similarities between books. By applying cosine similarity, the system calculates how closely related different books are based on user ratings. When a user selects or searches for a book, the system recommends similar books that match the user’s interests.

One of the key advantages of this approach is that it does not require detailed knowledge of the book content. Instead, it relies on user interaction data, making it efficient and scalable for large datasets. The system continuously improves its recommendations as more data is collected, making it dynamic and adaptive.

In addition to collaborative filtering, the system integrates Natural Language Processing (NLP) techniques to enhance functionality. NLP is used to generate concise summaries of books, allowing users to quickly understand the content. This feature significantly improves decisionmaking and reduces the time required to evaluate books.

The proposed system is designed to handle large datasets efficiently and provide accurate recommendations. It reduces the dependency on manual searching and enhances user experience by delivering personalized suggestions. The integration of machine learning and NLP makes the system more intelligent and user-centric.

5.2.1 ADVANTAGES OF PROPOSED SYSTEM

- Provides personalized recommendations based on user preferences
- Improves accuracy using collaborative filtering and cosine similarity
- Reduces manual effort in searching for books
- Adapts dynamically based on user interactions
- Integrates NLP for generating book summaries
- Enhances user experience and satisfaction
- Efficient and scalable for large datasets
- User-friendly web interface for easy interaction

SYSTEM DESIGN

6. SYSTEM DESIGN

6.1 INPUT DESIGN

Input Design plays a crucial role in the development of any system, as it determines how data is entered into the system and ensures that the data provided by the user is accurate, valid, and efficient for processing. The main objective of input design is to convert user inputs into a format that can be easily processed by the system while minimizing errors and ensuring data integrity.

In the proposed system, “Collaborative Filtering-Based Book Recommender System Using Machine Learning,” the input design is carefully structured to provide a smooth and userfriendly experience. The system includes multiple input interfaces where users can interact with the application.

The primary input to the system is provided through the user interface, where users can register and log in to access personalized features. During registration, users are required to enter details such as username, email, and password. These inputs are validated to ensure correctness, such as checking for proper email format and secure password entry.

Once logged in, users can provide input in the form of book names, preferences, or selections. The system may also consider user ratings and interactions as input data. These inputs are used by the recommendation engine to analyze user behavior and generate relevant suggestions.

The system also handles implicit inputs, such as user browsing history, clicked items, and interaction time. These inputs are automatically captured and used to improve recommendation accuracy over time.

Proper validation techniques are implemented to ensure that incorrect or incomplete data is not processed. For example, empty fields are not accepted, invalid inputs are rejected, and appropriate error messages are displayed to guide the user. This helps in maintaining data quality and preventing system errors.

The input design is structured in such a way that it reduces user effort and enhances usability. Forms are designed with clear labels and instructions, and the system provides feedback to users in case of incorrect input. Overall, the input design ensures that the data entered into the system is accurate, consistent, and suitable for further processing.

6.2 OUTPUT DESIGN

Output Design is an important aspect of system development that focuses on how the results are presented to the user. The main objective of output design is to provide meaningful, clear, and useful information to users in an understandable format.

In the proposed system, the output is primarily in the form of book recommendations and summaries. After processing the user input, the system generates a list of recommended books that match the user's preferences. These recommendations are displayed in a structured and visually appealing format on the user interface.

Each recommended book is presented with relevant details such as title, author, and rating. In addition to recommendations, the system also provides summaries of books generated using Natural Language Processing (NLP). These summaries help users quickly understand the content of the book without reading lengthy descriptions.

The output design ensures that the information is easy to read and interpret. The results are displayed clearly with proper formatting, making it convenient for users to explore recommendations. The system may also highlight similar books or related items to provide additional suggestions.

The system provides feedback messages such as successful login, registration confirmation, and error notifications. These messages help users understand the status of their actions and guide them through the system.

The output is designed to be user-friendly, interactive, and responsive. It ensures that users receive accurate and relevant information quickly, enhancing their overall experience.

Overall, the output design of the system plays a vital role in improving usability by presenting recommendations in a clear, concise, and informative manner.

ALGORITHMS

7. ALGORITHMS

In the proposed system, various machine learning and artificial intelligence techniques are used to generate personalized book recommendations and improve user experience. The system primarily uses collaborative filtering for recommendation generation, supported by similarity measurement techniques and Natural Language Processing (NLP) for content understanding. These algorithms work together to analyze user behavior, identify patterns, and provide accurate and meaningful suggestions.

7.1 COLLABORATIVE FILTERING

Collaborative filtering is the core algorithm used in the system for generating personalized recommendations. It is based on the principle that users who have similar preferences in the past are likely to have similar interests in the future.

The system collects user interaction data such as ratings and reviews and uses this information to identify relationships between users. Based on these relationships, the system recommends books that similar users have liked or rated highly.

In this project, user-based collaborative filtering is implemented. The system constructs a user-item interaction matrix where users are represented as rows and books as columns. Each value in the matrix represents the rating given by a user to a particular book.

The algorithm then compares users based on their rating patterns and identifies similar users. Books that are highly rated by these similar users are recommended to the target user.

Advantages of collaborative filtering:

- Does not require detailed knowledge of book content
- Provides personalized recommendations
- Improves over time as more data is collected

7.2 COSINE SIMILARITY

Cosine similarity is used to measure the similarity between users based on their rating vectors. It calculates the cosine of the angle between two vectors in a multi-dimensional space.

The similarity value ranges from 0 to 1:

- 1 indicates that users are highly similar
- 0 indicates that users are completely different

In this system, each user is represented as a vector of ratings for different books. Cosine similarity is applied to these vectors to determine how similar users are to each other.

Working steps:

- Convert user ratings into vector form
- Compute cosine similarity between users
- Generate a similarity matrix
- Select top similar users

Cosine similarity is efficient and suitable for sparse datasets, making it ideal for recommendation systems.

7.3 USER-BASED RECOMMENDATION ALGORITHM

The system uses a user-based recommendation approach as part of collaborative filtering. In this method, the system identifies a group of users whose preferences are similar to the current user.

Once similar users are identified using cosine similarity, the system retrieves the books that these users have rated highly. These books are then recommended to the current user.

Steps involved:

1. Identify similar users
2. Select top-N similar users
3. Extract highly rated books from these users
4. Recommend those books

This approach ensures that the recommendations are personalized and relevant to user interests.

7.4 POPULARITY-BASED RECOMMENDATION

The system also incorporates a popularity-based recommendation approach. This method recommends books based on their overall popularity, such as the number of ratings and average rating.

This approach is particularly useful in situations where there is insufficient data about a user, such as new users who have not provided any ratings. In such cases, the system recommends popular books that are widely accepted by other users.

Popularity is determined by analyzing:

- Number of ratings
- Average rating score

The top-ranked books are then suggested to users as default recommendations.

7.5 NATURAL LANGUAGE PROCESSING (NLP)

Natural Language Processing (NLP) is used in the proposed system to analyze textual data and generate concise summaries of books. It enables the system to process human language efficiently and extract meaningful information from book descriptions.

In this project, a transformer-based deep learning model is used for text summarization. The system utilizes a pre-trained model through the Hugging Face Transformers library, specifically the BART (Bidirectional and Auto-Regressive Transformer) model, which is designed for sequence-to-sequence learning tasks such as text summarization.

The NLP module improves user experience by providing short and meaningful summaries, allowing users to quickly understand the content of books without reading lengthy descriptions.

By integrating these techniques, the system achieves higher accuracy, better performance, and improved user experience.

SAMPLE CODE

8 . SAMPLE CODE

app.py

```
from flask import Flask, render_template, request, redirect, url_for, session, flash,
send_from_directory, jsonify
import pandas as pd
import sqlite3
import os
import re
import requests
import numpy as np
from werkzeug.security import generate_password_hash, check_password_hash
from werkzeug.utils import secure_filename
from sklearn.metrics.pairwise import cosine_similarity
from flask_mail import Mail, Message
from itsdangerous import URLSafeTimedSerializer
from nlp_summary import multilingual_summary

app = Flask(__name__)
app.secret_key = "bloomverse_secret_key"

serializer = URLSafeTimedSerializer(app.secret_key)

app.config["MAIL_SERVER"] = "smtp.gmail.com"
app.config["MAIL_PORT"] = 587
app.config["MAIL_USE_TLS"] = True
app.config["MAIL_USERNAME"] = "chikkalabhagyalakshmi65@gmail.com"
app.config["MAIL_PASSWORD"] = "euehqkbyknjdgo"

mail = Mail(app)

UPLOAD_FOLDER = "static/profile_images"
app.config["UPLOAD_FOLDER"] = UPLOAD_FOLDER

if not os.path.exists(UPLOAD_FOLDER):
    os.makedirs(UPLOAD_FOLDER)

def init_db():
    with sqlite3.connect("users.db") as conn:
        c = conn.cursor()

        c.execute("""
            CREATE TABLE IF NOT EXISTS users (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                username TEXT,
                email TEXT UNIQUE,
```

```
        phone TEXT,  
        password TEXT,  
        profile_image TEXT,  
        role TEXT DEFAULT 'user'  
    )  
""")
```

```
c.execute("""  
    CREATE TABLE IF NOT EXISTS wishlist (  
        id INTEGER PRIMARY KEY AUTOINCREMENT,  
        username TEXT,  
        book_title TEXT,  
        author TEXT,  
        image TEXT  
    )  
""")
```

```
c.execute("""  
    CREATE TABLE IF NOT EXISTS user_preferences (  
        id INTEGER PRIMARY KEY AUTOINCREMENT,  
        user_email TEXT UNIQUE,  
        favorite_author TEXT,  
        favorite_genre TEXT,  
        favorite_language TEXT  
    )  
""")
```

```
c.execute("""  
    CREATE TABLE IF NOT EXISTS search_history (  
        id INTEGER PRIMARY KEY AUTOINCREMENT,  
        user_email TEXT,  
        query TEXT,  
        searched_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
    )  
""")
```

```
c.execute("""  
    CREATE TABLE IF NOT EXISTS reviews (  
        id INTEGER PRIMARY KEY AUTOINCREMENT,  
        user_email TEXT,  
        book_title TEXT,  
        rating INTEGER,  
        review TEXT,  
        likes INTEGER DEFAULT 0  
    )  
""")
```

```

c.execute("""
    CREATE TABLE IF NOT EXISTS recently_viewed (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        user_email TEXT,
        book_title TEXT,
        viewed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    )
""")

conn.commit()

init_db()

books = pd.read_csv("datasets/books_multilingual_final_ready.csv")
books = books.dropna(subset=["title", "author"])

COVER_FOLDER = "static/book_covers"

def get_cover(row):
    if "img" in row and isinstance(row["img"], str) and row["img"].startswith("http"):
        return row["img"]

    if "image" in row and isinstance(row["image"], str) and row["image"].startswith("http"):
        return row["image"]

    title = row["title"]
    clean = re.sub(r"^[^a-z0-9]+", "_", title.lower()).strip("_")

    possible = [
        f"{clean}.jpg",
        f"{clean}_telugu.jpg",
        f"{clean}_tamil.jpg",
        f"{clean}_hindi.jpg",
        f"{clean}_kannada.jpg",
        f"{clean}_malayalam.jpg"
    ]

    for file in possible:
        full = os.path.join(COVER_FOLDER, file)
        if os.path.exists(full):
            return f"book_covers/{file}"

    return "book_covers/default.jpg"

books["img"] = books.apply(get_cover, axis=1)
IMG_COLUMN = "img"

```

```

avoid_genres = ["Romance", "Paranormal", "Erotica", "Young Adult Romance"]

def filter_academic_books(df):
    if "genre" in df.columns:
        return df[~df["genre"].str.contains("|".join(avoid_genres), case=False, na=False)]
    return df

def get_google_books_data(title, author=None):
    try:
        url = "https://www.googleapis.com/books/v1/volumes"
        query = title if not author else f'{title}+inauthor:{author}'

        response = requests.get(url, params={"q": query, "maxResults": 1}, timeout=5)
        data = response.json()

        if "items" in data:
            volume = data["items"][0]["volumeInfo"]
            return {
                "preview": volume.get("previewLink"),
                "description": volume.get("description")
            }

    except Exception:
        pass

    return {}

def get_amazon_link(title):
    return f"https://www.amazon.in/s?k={title.replace(' ', '+')}}"

def get_openlibrary_link(title):
    return f"https://openlibrary.org/search?q={title.replace(' ', '+')}}"

def get_recommendations(user_email):
    with sqlite3.connect("users.db") as conn:
        df = pd.read_sql_query("SELECT user_email, book_title, rating FROM reviews", conn)

    if df.empty:
        return filter_academic_books(books).sample(10).to_dict(orient="records")

    pivot = df.pivot_table(index="user_email", columns="book_title", values="rating").fillna(0)

    if pivot.shape[0] < 2:
        return filter_academic_books(books).sample(10).to_dict(orient="records")

    similarity = cosine_similarity(pivot)

```

```

sim_df = pd.DataFrame(similarity, index=pivot.index, columns=pivot.index)

if user_email not in sim_df.index:
    return filter_academic_books(books).sample(10).to_dict(orient="records")

similar_users = sim_df[user_email].sort_values(ascending=False)[1:4].index
recommended_titles = df[df.user_email.isin(similar_users)].book_title.unique()

result = books[books.title.isin(recommended_titles)]

if result.empty:
    return filter_academic_books(books).sort_values(by="rating",
ascending=False).head(10).to_dict(orient="records")

    return filter_academic_books(result).head(10).to_dict(orient="records")

@app.route("/")
def index():
    return render_template("index.html")

@app.route("/register", methods=["GET", "POST"])
def register():
    if request.method == "POST":

        username = request.form.get("username")
        email = request.form.get("email")
        phone = request.form.get("phone")
        password = request.form.get("password")
        confirm_password = request.form.get("confirm_password")

        if password != confirm_password:
            flash("Passwords do not match!")
            return redirect(url_for("register"))

        hashed_password = generate_password_hash(password)

        try:
            with sqlite3.connect("users.db") as conn:
                c = conn.cursor()
                c.execute(
                    "INSERT INTO users (username, email, phone, password) VALUES (?, ?, ?, ?)",
                    (username, email, phone, hashed_password)
                )
                conn.commit()

        except sqlite3.IntegrityError:
            flash("Email already exists!")

```

```

        return redirect(url_for("register"))

    session["user"] = username
    session["email"] = email

    return redirect(url_for("explore"))

return render_template("register.html")

@app.route("/user_login", methods=["GET", "POST"])
def user_login():
    if request.method == "POST":

        email = request.form.get("email")
        password = request.form.get("password")

        with sqlite3.connect("users.db") as conn:
            c = conn.cursor()
            c.execute("SELECT * FROM users WHERE email=?", (email,))
            user = c.fetchone()

        if user and check_password_hash(user[4], password):
            session["user"] = user[1]
            session["email"] = user[2]
            return redirect(url_for("explore"))

        flash("Invalid credentials!")

    return render_template("user_login.html")

@app.route("/explore")
def explore():
    if "user" not in session:
        return redirect(url_for("user_login"))

    recommended = filter_academic_books(books).head(12)

    return render_template(
        "explore.html",
        recommended=recommended.to_dict(orient="records")
    )

if __name__ == "__main__":
    app.run(debug=True)

```

SYSTEM TESTING

9. SYSTEM TESTING

The purpose of testing is to discover errors and ensure that the “Collaborative Filtering-Based Book Recommender System Using Machine Learning” works correctly according to user requirements. Testing is the process of evaluating the application to identify faults, validate functionality, and ensure that the system performs efficiently without failures. It ensures that all modules such as user authentication, book recommendation, NLP-based summary generation, search functionality, wishlist management, and review system work properly. Various types of testing are performed to verify the performance, reliability, and accuracy of the system.

9.1 TESTING STRATEGIES:

9.1.1 UNIT TESTING

Unit testing involves testing individual modules of the system to ensure that each component functions correctly. Each module such as login/register, recommendation engine, NLP summary generation, search, wishlist, and review system is tested separately.

Test objectives:

- All input fields such as username, email, password, search query, and ratings must work properly.
- Each module should produce correct outputs based on given inputs.
- Error messages should be displayed for invalid inputs.
- Data processing functions like recommendation logic and NLP summarization should work correctly.

Features to be tested:

- Verification of input formats (email validation, password validation).
- Checking whether recommendations are generated correctly using collaborative filtering.
- Ensuring no duplicate user or data entries in database.
- Validating navigation between pages such as login, dashboard, and book details.

9.1.2 INTEGRATION TESTING

Integration testing is performed to ensure that all modules of the system work together correctly. After testing individual components, modules such as frontend interface, backend processing, database, recommendation engine, and NLP module are integrated and tested as a whole.

The following are the types of Integration Testing:

Top Down Integration

In this method, testing starts from the main modules such as homepage and dashboard and proceeds to sub-modules like book recommendation, search, and summary generation. This ensures that the complete workflow of the system is functioning correctly.

Bottom-up Integration

In this method, testing begins with lower-level components such as database operations, cosine similarity calculation, and NLP summarization functions. These components are tested first and then integrated with higher-level modules such as the user interface and web application.

9.1.3 FUNCTIONAL TESTING

Functional testing ensures that all functionalities of the system operate according to the specified requirements. Each feature is tested with both valid and invalid inputs to verify correct behavior.

Functional testing is centered on the following items:

Valid Input:

User inputs such as login details, search queries, ratings, and book selections should be accepted.

Invalid Input:

Incorrect inputs such as empty fields, invalid email formats, or wrong credentials should be rejected.

Functions:

All modules like recommendation system, search functionality, NLP summary generation, wishlist, and review system must function properly.

Output:

The system should display accurate book recommendations, search results, and summaries.

Systems/Procedures:

All integrated processes such as login, email notifications, database storage, and recommendation generation should function correctly.

9.2 SYSTEM TESTING TECHNIQUES:

System testing ensures that the complete application works as expected in real-time conditions. It validates the full workflow including login, book exploration, recommendation generation, summary display, and user interactions.

9.2.1 WHITE BOX TESTING

White box testing is performed with knowledge of the internal structure and code of the system. It is used to test backend logic such as collaborative filtering algorithm, cosine similarity calculations, NLP summarization process, and database operations.

This testing ensures:

- Correct working of recommendation algorithms
- Proper execution of similarity calculations
- Accurate data retrieval and storage
- Efficient functioning of internal logic

9.2.2 BLACK BOX TESTING

Black box testing is performed without knowledge of the internal implementation of the system. The application is tested from the user's perspective by providing inputs and verifying outputs.

This testing ensures:

- Correct response to user inputs
- Proper display of recommendations and summaries
- User-friendly interface and interaction

SOFTWARE ENVIRONMENT

10. SOFTWARE ENVIRONMENT

10.1 Machine Learning

Tom Mitchell defines Machine Learning as “A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E.” Machine Learning focuses on identifying patterns and relationships within datasets and using them to make predictions or generate meaningful insights.

In the proposed system, machine learning is used to generate personalized book recommendations based on user preferences and ratings. The system applies collaborative filtering and cosine similarity techniques to identify similar users and recommend books accordingly. Machine learning plays a significant role in improving recommendation accuracy by continuously learning from user interactions and historical data.

Machine learning tasks are generally classified into supervised learning, unsupervised learning, semi-supervised learning, and reinforcement learning, depending on the type of data and learning approach used.

10.1.1 Benefits of Machine Learning

Machine learning offers several advantages that improve the overall performance and efficiency of the system:

- Improves accuracy of book recommendations based on user behavior
- Reduces manual effort by automating the recommendation process
- Provides personalized user experience
- Handles large datasets efficiently
- Identifies hidden patterns in user preferences
- Enhances system scalability and flexibility
- Supports predictive analysis and decision-making
- Reduces processing time and increases efficiency

10.1.2 Features of Machine Learning

Machine learning provides powerful capabilities that enhance system intelligence:

- Automates decision-making processes
- Learns from data without explicit programming

- Improves performance over time with new data
- Uses algorithms and statistical models for predictions
- Supports pattern recognition and classification
- Handles complex and high-dimensional data
- Enables dynamic and adaptive systems

10.2 Introduction to Python

Python is a high-level, interpreted, and object-oriented programming language widely used for web development, data analysis, and machine learning applications. It is designed to be simple, readable, and efficient, making it suitable for both beginners and experienced developers.

Python supports multiple programming paradigms and is processed at runtime, which makes debugging easier. It provides a rich set of libraries and frameworks that simplify development and enable rapid implementation of complex systems.

10.2.1 Features of Python

Python offers several features that make it suitable for this project:

- Easy to learn and use
- Simple and readable syntax
- Interpreted language (no compilation required)
- Object-oriented programming support
- Cross-platform compatibility
- Extensive standard library
- Strong support for machine learning and data analysis
- Easy integration with databases

10.2.2 Benefits of Python

Python provides multiple benefits for system development:

- Reduces development time due to simple syntax
- Improves productivity and efficiency
- Provides powerful libraries for machine learning
- Supports rapid prototyping
- Easy debugging and testing
- Large community support

- Scalable and flexible for various applications

10.3 Flask

Flask is a lightweight and flexible Python web framework used for building web applications. It is designed to be simple and does not enforce strict project structure, allowing developers to build applications according to their needs.

Flask uses a routing mechanism where URLs are mapped to functions that handle user requests. It supports template rendering, session management, and easy integration with databases and machine learning models. Flask is widely used for developing scalable and efficient web applications.

10.3.1 Features of Flask

Flask provides several features that support web application development:

- Lightweight and minimal framework
- Flexible and customizable architecture
- Built-in development server
- URL routing system
- Template rendering using Jinja2
- Session and request handling
- Easy integration with databases like SQLite
- Supports integration with machine learning models

10.3.2 Benefits of Flask

Flask offers various advantages for developers:

- Simple and easy to learn
- Faster development with minimal configuration
- Flexible project structure
- Lightweight compared to other frameworks
- Easy integration with Python libraries
- Suitable for small to medium-scale applications
- Supports rapid development and testing

SYSTEM DIAGRAMS

11. SYSTEM DIAGRAMS

11.1 SYSTEM ARCHITECTURE

A system architecture or systems architecture is the conceptual model that defines the structure, behavior, and more views of a system. An architecture description is a formal description and representation of a system. Organized in a way that supports reasoning about the structures and behaviors of the system

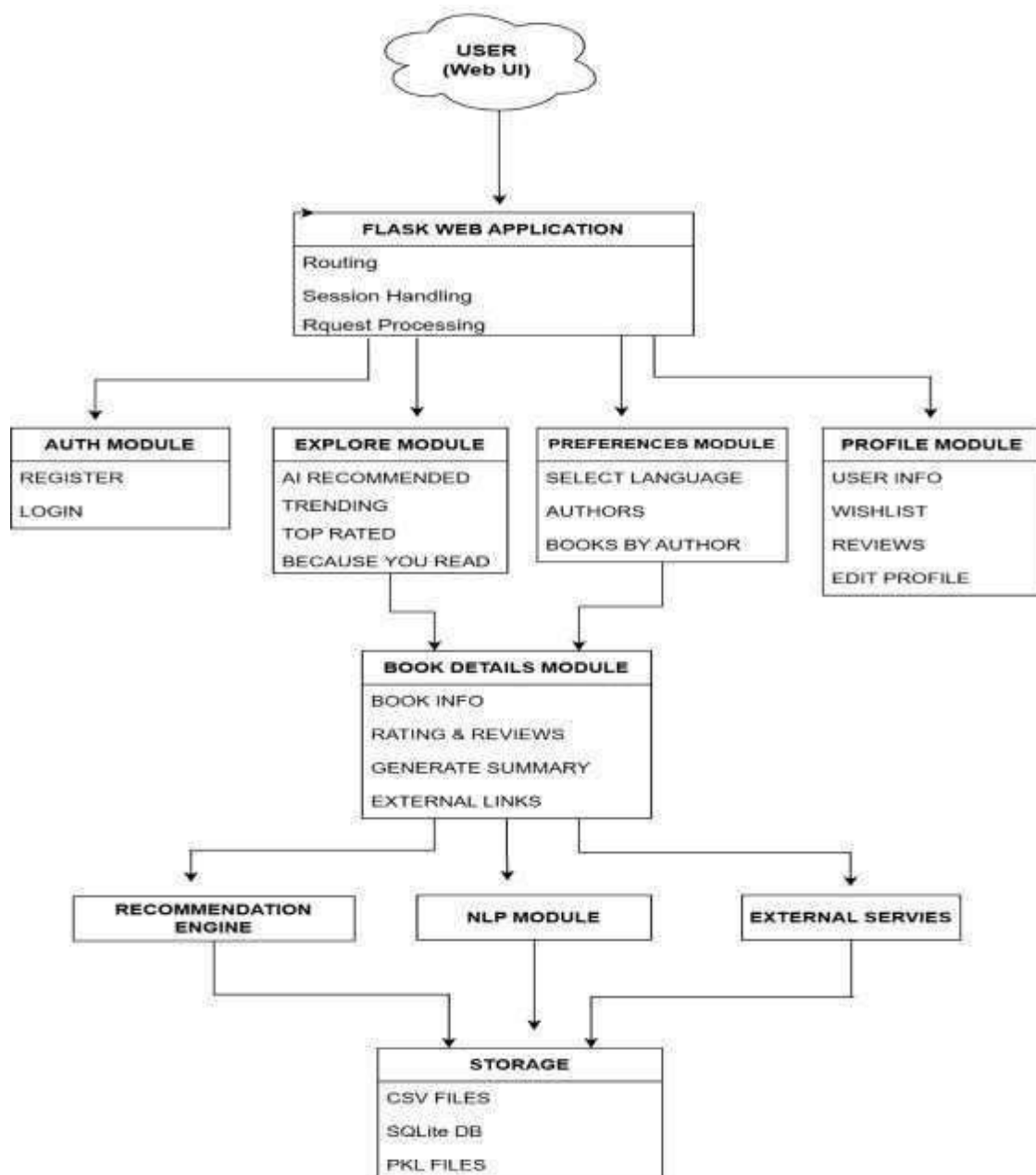


FIG 11.1 ARCHITECTURE DIAGRAM

11.2 WORK FLOW DIAGRAM

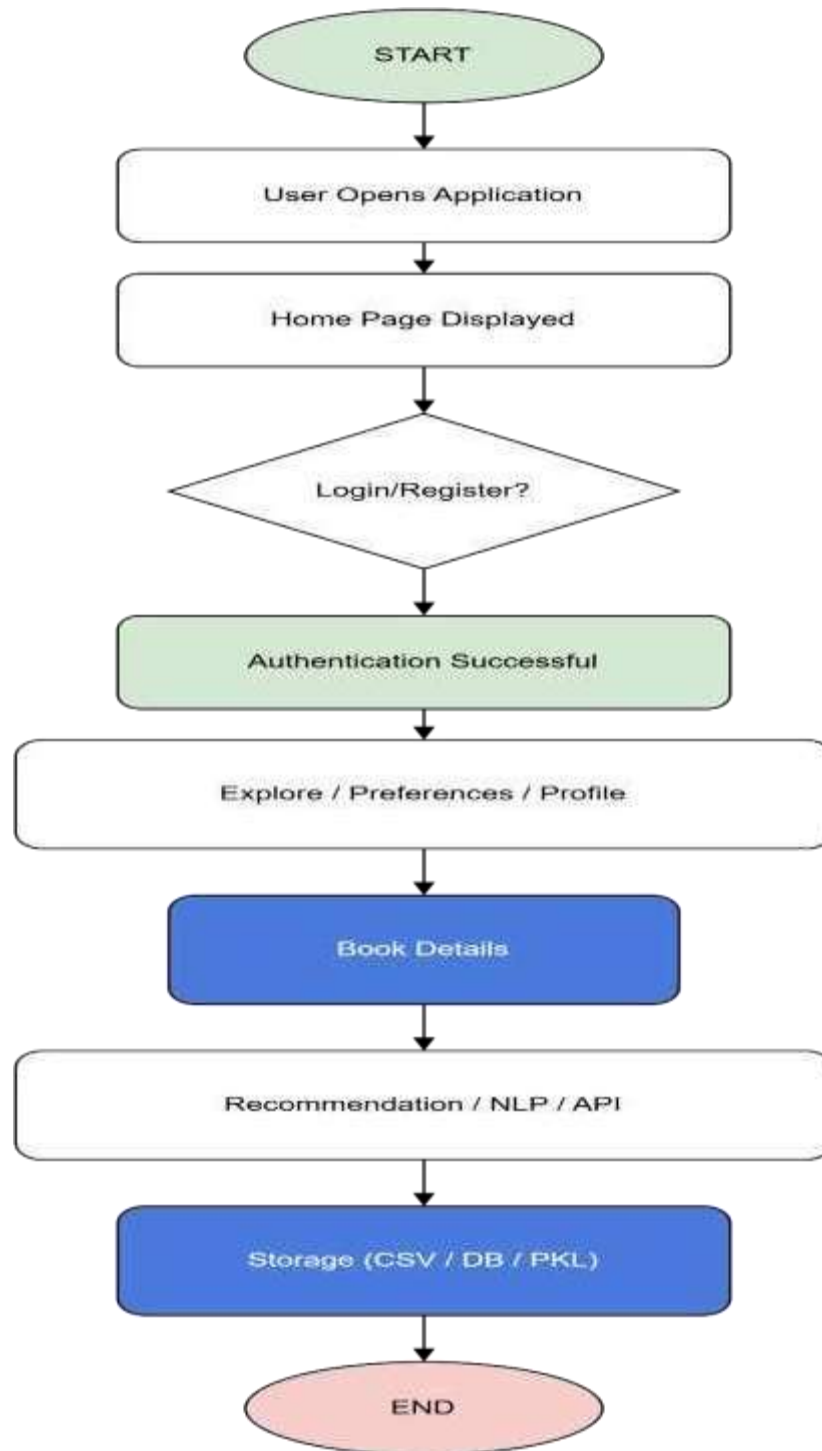


FIG 11.2 WORK FLOW DIAGRAM

11.3 UML DIAGRAMS:

UML (Unified Modeling Language) is a standardized, general-purpose visual modeling language used in the field of Software Engineering. It is used for specifying, visualizing, constructing, and documenting the primary artifacts of a software system.

It helps in designing and representing systems, especially those based on object-oriented concepts. UML describes the working of both software and hardware systems.

UML was developed during 1994–1995 by Grady Booch, Ivar Jacobson, and James Rumbaugh at Rational Software. In 1997, it was adopted as a standard by the Object Management Group (OMG).

The Object Management Group (OMG) is an association of companies that manages and maintains the UML standard. It was established to create open standards that support interoperability of object-oriented systems.

UML is not limited to software systems; it can also be used to model non-software systems. The OMG is also known for developing the Common Object Request Broker Architecture (CORBA) standards.

11.3.1 USE CASE DIAGRAM

To model a system effectively, it is important to capture its dynamic behavior. Dynamic behavior refers to how the system operates during execution.

Only representing static behavior is not sufficient; understanding how the system behaves during runtime is more important.

In a system, both internal and external entities that interact with it are called actors.

A use case diagram consists of:

- Actors
- Use Cases
- Relationships between them

It is used to model the functionality of a system or subsystem. Each use case diagram represents a specific functionality of the system and shows how users interact with the system.

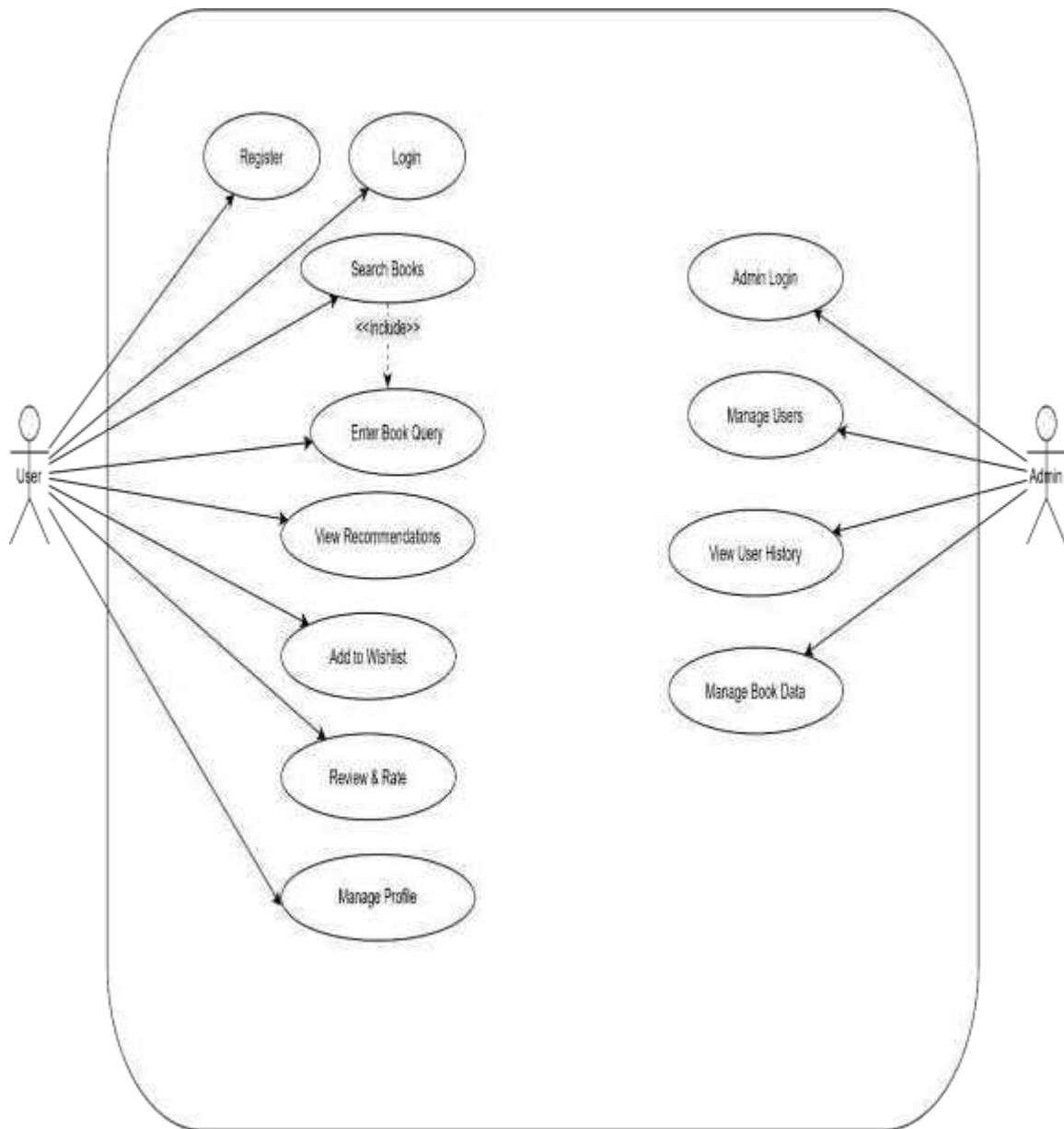


FIG 11.3.1 USE CASE DIAGRAM

11.3.3 ACTIVITY DIAGRAM

An Activity Diagram is a UML diagram used to represent the flow of activities or actions in a system.

It shows how different processes are executed step-by-step, including decisions, loops, and parallel tasks.

It helps in understanding the workflow and behavior of a system in a clear and visual way.

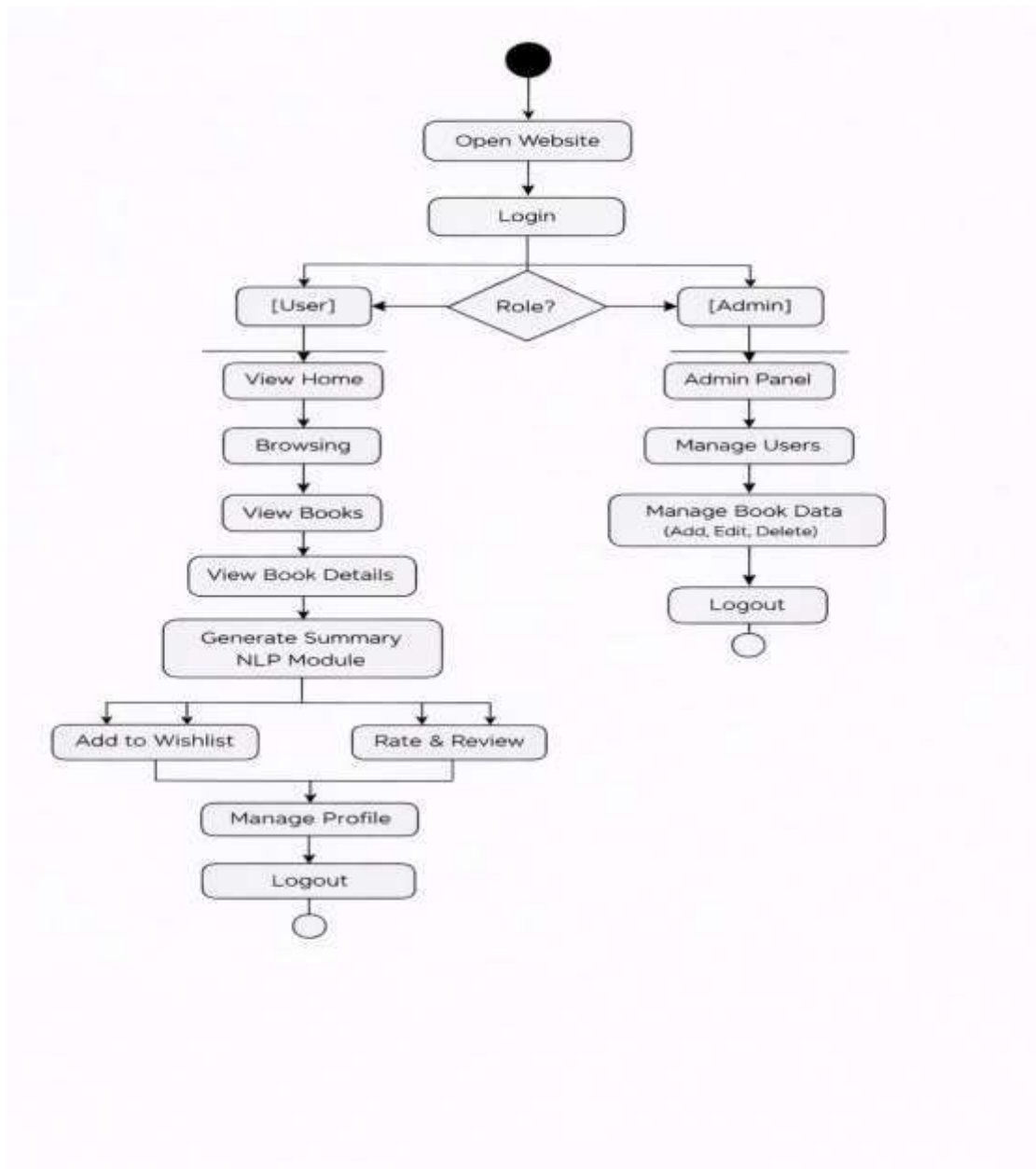


FIG 11.3.3 ACTIVITY DIAGRAM

11.3.4 STATE MACHINE DIAGRAM

A State Machine Diagram is a UML diagram that shows the different states of an object and how it changes from one state to another.

It represents events, conditions, and transitions that control the behavior of the system. It is useful for modeling dynamic behavior, especially in systems where states change based on inputs or events.

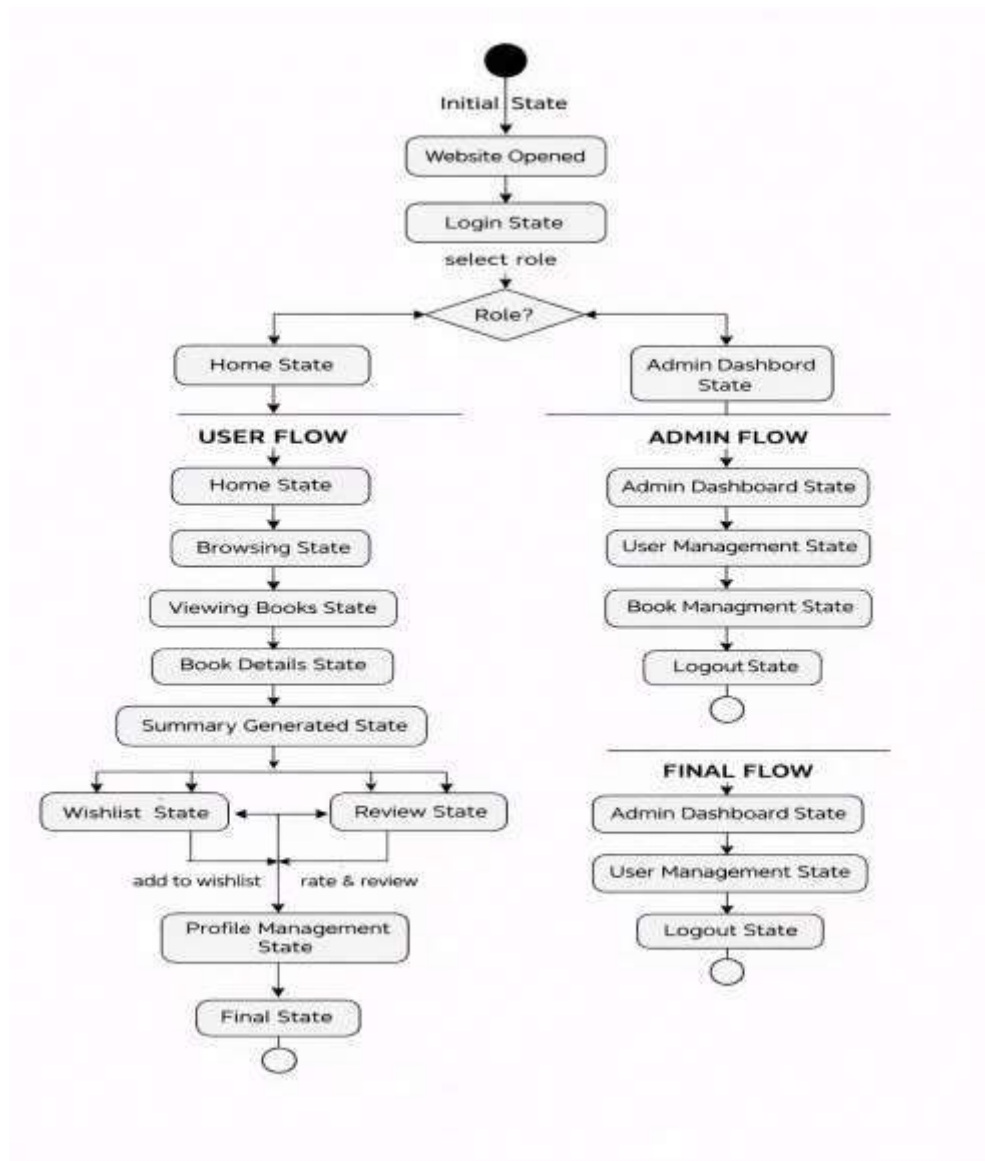


FIG 11.3.4 STATE MACHINE DIAGRAM

11.3.5 SEQUENCE DIAGRAM

A Sequence Diagram is a UML diagram that shows how objects interact with each other in a specific order over time.

It represents the sequence of messages exchanged between different components in a system. It helps in understanding the flow of communication and execution of processes step-by-step.

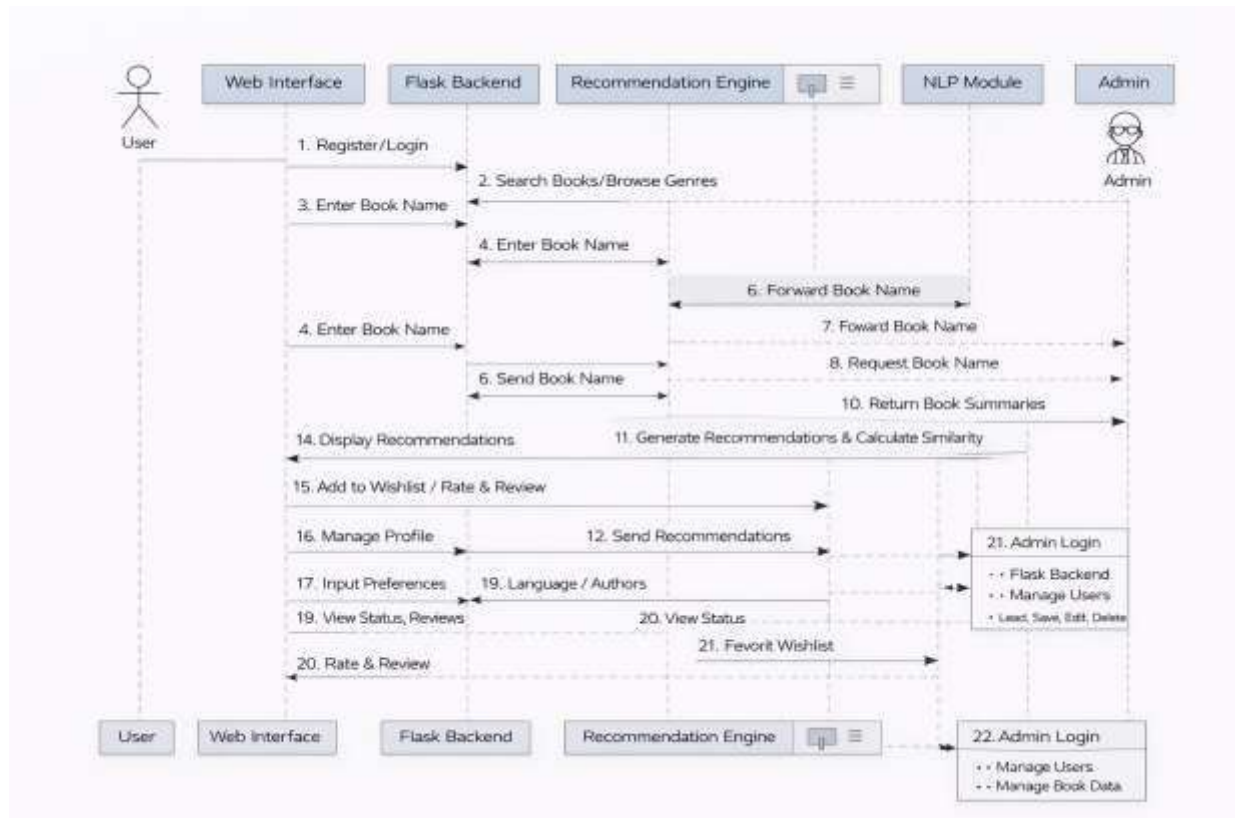


FIG 11.3.5 SEQUENCE DIAGRAM

SYSTEM IMPLEMENTATION

12. SYSTEM IMPLEMENTATION

12.1 INTRODUCTION

System implementation is the phase where the designed system is developed and made operational. It involves coding, integration of different modules, and testing to ensure that the system functions as expected. The implementation of the “Collaborative Filtering-Based Book Recommender System Using Machine Learning” is carried out using Python, Flask framework, and various machine learning libraries.

The system is implemented as a web-based application where users can interact through a browser. The frontend is developed using HTML, CSS, and JavaScript, while the backend is implemented using Flask. The database is managed using SQLite, which stores user data, reviews, wishlist, and other information.

12.2 MODULES IMPLEMENTED

The system is divided into multiple modules, each responsible for a specific functionality. These modules are integrated to form a complete application.

1. User Authentication Module

This module handles user registration and login functionality. Users can create an account by providing their details such as username, email, and password. Passwords are stored securely using hashing techniques. The login system verifies user credentials and allows access to the application.

2. Recommendation Module

This module is the core part of the system. It generates personalized book recommendations using collaborative filtering and cosine similarity. The system analyzes user ratings and identifies similar users. Based on their preferences, relevant books are recommended to the user.

3. NLP Summarization Module

This module uses Natural Language Processing techniques to generate summaries of book descriptions. A transformer-based model (BART) is used to process textual data and generate concise summaries. This helps users understand the content of books quickly.

4. Search Module

The search module allows users to find books based on keywords. It processes user queries and retrieves matching results from the dataset. This feature improves user experience by enabling quick access to books.

5. Review and Wishlist Module

This module allows users to add reviews and ratings for books. Users can also add books to their wishlist for future reference. The data is stored in the database and used to improve recommendations.

6. Admin Module

The admin module allows administrators to manage users and monitor system activity. Admins can view user details and perform management operations.

12.3 IMPLEMENTATION TOOLS AND TECHNOLOGIES

The system is implemented using the following tools and technologies:

- Python for backend logic and machine learning
- Flask framework for web application development
- HTML, CSS, JavaScript for frontend design
- SQLite for database management
- Scikit-learn for implementing cosine similarity
- Transformers library for NLP summarization

12.4 IMPLEMENTATION PROCESS

The implementation process involves several steps. Initially, the dataset is collected and preprocessed using Python libraries such as Pandas and NumPy. The recommendation model is then developed using collaborative filtering techniques.

Next, the Flask application is developed to handle user requests and integrate all modules. The database is created to store user information and interactions. The NLP module is integrated to generate summaries. Finally, all modules are combined and tested to ensure proper functionality.

12.5 SYSTEM WORKFLOW

The system workflow describes how the application operates:

- User registers and logs into the system
- User searches for books or interacts with the application
- User ratings and interactions are stored in the database
- Recommendation engine processes data and generates suggestions
- NLP module generates summaries of books
- Results are displayed to the user

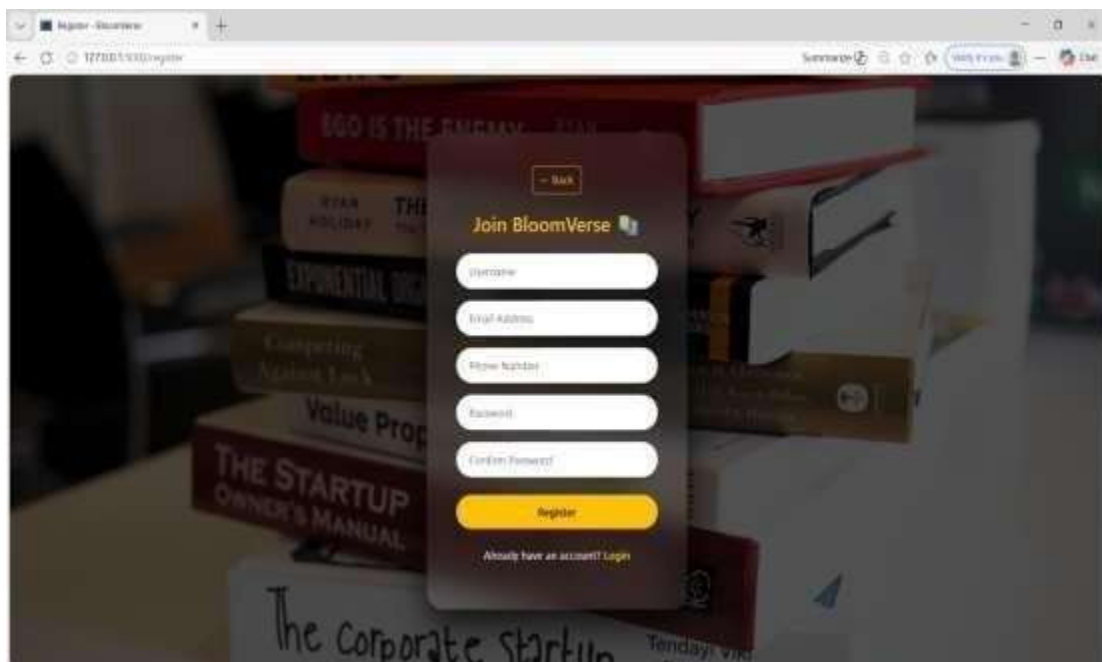
SCREEN SHOTS

13. SCREEN SHOTS

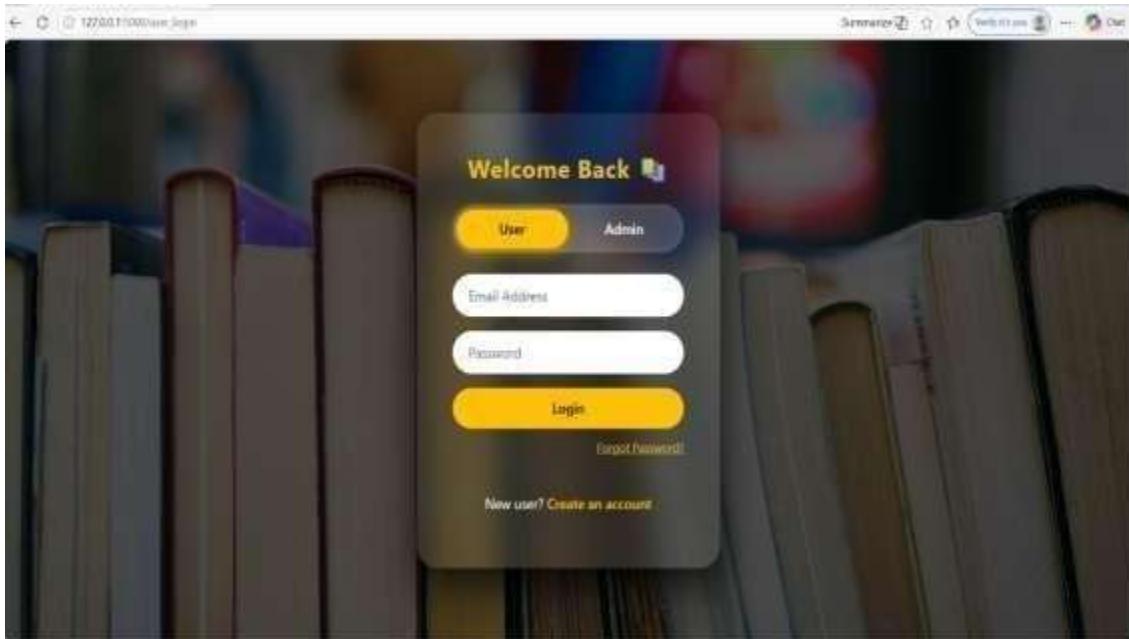
13.2 HOME PAGE OUTPUT



13.2 USER REGISTRATION PAGE OUTPUT



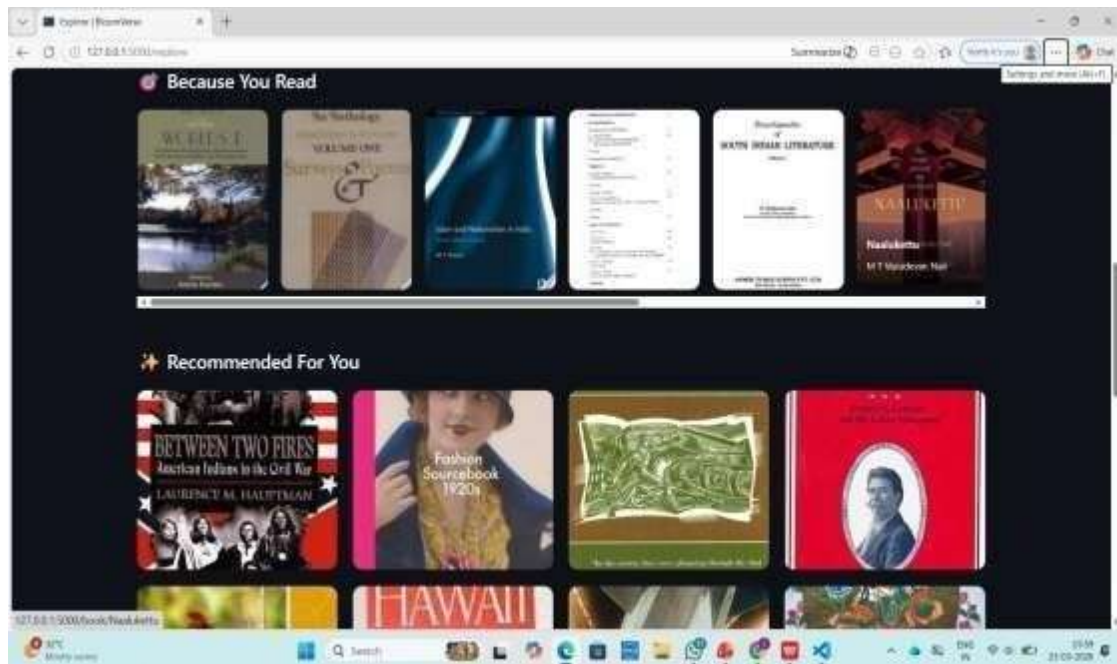
13.3 USER & ADMIN LOGIN PAGE OUTPUT



13.4 SEARCH FUNCTIONALITY OUTPUT



13.5 BOOK RECOMMENDATION OUTPUT



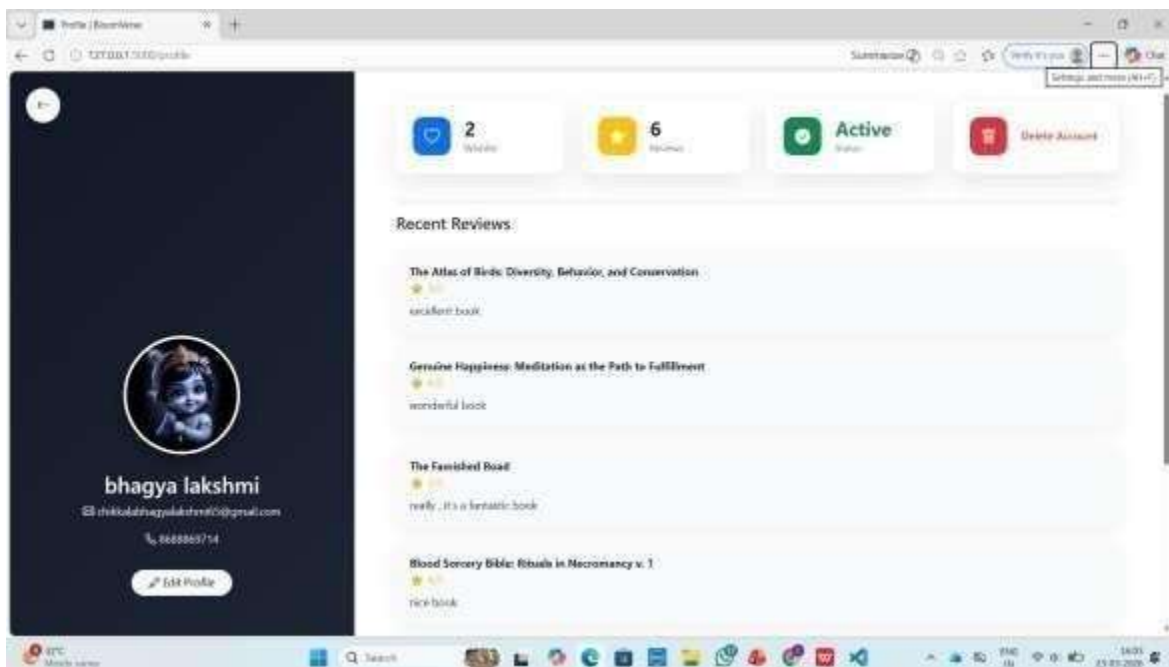
13.6 BOOK DETAILS & SUMMARY OUTPUT



13.7 WISHLIST PAGE OUTPUT



13.8 PROFILE PAGE OUTPUT



CONCLUSION

14. CONCLUSION

The “Collaborative Filtering-Based Book Recommender System Using Machine Learning” is designed and developed to provide personalized and efficient book recommendations to users. The system successfully integrates machine learning techniques, web technologies, and Natural Language Processing to enhance user experience and improve recommendation accuracy.

The implementation of collaborative filtering using cosine similarity enables the system to analyze user preferences and generate relevant recommendations based on similar users. This approach ensures that users receive suggestions that match their interests without requiring detailed knowledge of book content. Additionally, the integration of NLP using a transformer-based model helps in generating concise summaries, allowing users to quickly understand the content of books.

The system also includes essential features such as user authentication, search functionality, review and rating system, wishlist management, and admin controls. These features contribute to making the application interactive, user-friendly, and efficient. The use of Flask as a backend framework and SQLite as a database ensures smooth performance and easy deployment.

Overall, the developed system achieves its objective of providing accurate and personalized recommendations while maintaining simplicity and efficiency. It reduces the effort required by users to search for suitable books and enhances decision-making through intelligent suggestions.

FUTURE ENHANCEMENTS

15. FUTURE ENHANCEMENTS

The proposed system provides an efficient and personalized book recommendation platform; however, it can be further enhanced by incorporating advanced technologies and additional features to improve accuracy, scalability, and user experience.

One of the major improvements can be the implementation of hybrid recommendation systems that combine collaborative filtering with content-based filtering. This approach can overcome limitations such as the cold start problem and data sparsity, thereby improving the overall accuracy of recommendations.

The system can also be enhanced by integrating deep learning techniques such as neural collaborative filtering and recurrent neural networks. These models can capture complex patterns in user behavior and provide more accurate and dynamic recommendations. Additionally, real-time learning mechanisms can be implemented so that the system updates recommendations instantly based on user interactions.

Another important enhancement is the integration of real-time APIs such as Google Books API or other book databases. This would allow the system to fetch updated book information, including ratings, reviews, and descriptions, thereby improving the quality and relevance of recommendations.

The user interface can be further improved by incorporating modern UI/UX design principles. Features such as personalized dashboards, dark mode, interactive animations, and better navigation can enhance user engagement and usability. The system can also be developed as a mobile application to provide accessibility across different devices.

Voice-based interaction can be added using speech recognition and text-to-speech technologies. This would allow users to search for books, receive recommendations, and interact with the system using voice commands, making the system more user-friendly and accessible.

Another enhancement includes implementing advanced user profiling and analytics. By analyzing user behavior, preferences, and interaction patterns, the system can provide more precise and customized recommendations. Data visualization techniques can also be used to present insights in a clear and understandable manner.

The system can be scaled to handle large datasets and multiple users by deploying it on cloud platforms such as AWS, Azure, or Google Cloud. Cloud deployment will improve system performance, scalability, and availability. Additionally, the use of distributed databases can enhance data management and processing efficiency.

Security can also be strengthened by implementing advanced authentication mechanisms such as multi-factor authentication and encryption techniques to protect user data. Privacy-preserving machine learning techniques can be incorporated to ensure secure handling of sensitive information.

In addition to the above improvements, the system can be enhanced by incorporating explainable artificial intelligence (XAI) techniques. This will allow the system to provide clear explanations for its recommendations, such as showing why a particular book was suggested based on user preferences or similar user behavior. This transparency can increase user trust and improve the overall acceptance of the system. Furthermore, incorporating feedback

mechanisms where users can like, dislike, or refine recommendations will help the model learn more effectively and continuously improve its performance.

Another possible enhancement is the integration of multilingual support and cross-domain recommendations. By supporting multiple languages, the system can cater to a wider audience and improve accessibility for users from different regions. Cross-domain recommendation capabilities can also be introduced, where the system suggests related content such as articles, audiobooks, or educational resources based on user interests. These advancements will make the system more versatile, inclusive, and capable of providing a richer and more engaging user experience.

Furthermore, the system can be extended to support multiple domains such as movies, music, and e-commerce products, making it a generalized recommendation platform. Social features such as user communities, reviews, and sharing recommendations can also be added to increase user engagement.

Overall, these enhancements will make the system more intelligent, scalable, secure, and userfriendly, thereby improving its effectiveness and applicability in real-world scenarios.

REFERENCES

16. REFERENCES

- [1] G. Linden, B. Smith, and J. York, “Amazon.com recommendations: Item-to-item collaborative filtering,” *IEEE Internet Computing*, vol. 7, no. 1, pp. 76–80, Jan. 2003.
- [2] J. L. Herlocker, J. A. Konstan, and J. Riedl, “Evaluating collaborative filtering recommender systems,” *ACM Transactions on Information Systems*, vol. 22, no. 1, pp. 5–53, Jan. 2004.
- [3] Y. Koren, R. Bell, and C. Volinsky, “Matrix factorization techniques for recommender systems,” *IEEE Computer*, vol. 42, no. 8, pp. 30–37, Aug. 2009.
- [4] X. Su and T. M. Khoshgoftaar, “A survey of collaborative filtering techniques,” *Advances in Artificial Intelligence*, vol. 2009, Article ID 421425, 2009.
- [5] J. Bobadilla, F. Ortega, A. Hernando, and A. Gutiérrez, “Recommender systems survey,” *Knowledge-Based Systems*, vol. 46, pp. 109–132, 2013.